

Messy Mashup - Music Genre Classification

Deep Learning & Generative AI Project Report

Roll No: 24f1000556 | Term: T1 2026 | IIT Madras BS in Data Science

1. Abstract

This report covers my work on the Messy Mashup Kaggle competition, which involved classifying music into 10 genres under noisy, real-world conditions. The core challenge was a distribution shift between training and test data - training data consisted of clean instrument stems while test samples were mashups created by mixing stems from different songs, with tempo adjustments and environmental noise added on top. I built three models: a CNN from scratch, a CRNN (CNN + LSTM), and an Audio Spectrogram Transformer (AST) fine-tuned from a pretrained AudioSet checkpoint. The final approach used an ensemble of three AST models trained with different random seeds, achieving a Macro F1 score of 0.87237 on the Kaggle public leaderboard.

2. Introduction

2.1 Problem Statement

The Messy Mashup competition required building a model that can correctly identify the genre of a piece of music even when the audio has been significantly altered. Each test sample is a mashup constructed by mixing instrument stems (drums, bass, vocals, others) taken from different songs of the same genre. On top of that, random noise clips from the ESC-50 environmental sound dataset are added at varying positions and intensities. The model needs to identify the genre despite these distortions.

What makes this harder than standard genre classification is the distribution shift. During training, the data is clean and organized by genre and song. But at test time, the audio looks completely different. No single song's identity is preserved - the drums might be from one track, the vocals from another, all mixed with street noise. A model that just memorizes song-specific patterns will fail completely.

2.2 Project Goals

- Build at least three distinct model architectures including one from scratch and one pretrained
- Design augmentation strategies that bridge the train-test distribution gap
- Track all experiments with Weights & Biases for proper comparison
- Achieve a Macro F1 score above the 0.80 cutoff on Kaggle
- Deploy the final model as a working web application on Hugging Face Spaces

3. Dataset and Preprocessing

3.1 Dataset Description

The dataset has four main components:

Component	Description	Size
genres_stems/	10 genres x 100 songs x 4 stems (drums, bass, vocals, others)	4000 audio files
ESC-50-master/	Environmental noise clips across 50 classes	2000 clips
mashups/	Unlabeled noisy test mashups	3020 files
test.csv	ID to filename mapping for test set	3020 rows

3.2 Exploratory Data Analysis

I looked at the class distribution first and confirmed it was perfectly balanced - exactly 100 songs per genre across all 10 classes. No class imbalance to worry about, which is why Macro F1 and accuracy give similar signals on the validation set.

I also visualized Mel spectrograms for the vocal stem of one song per genre. A few patterns stood out right away. Metal and rock had dense energy across the full frequency range because of distorted guitars and heavy percussion. Classical had a lot of quiet gaps - much more dynamic variation than the other genres. Hip-hop and reggae both showed strong low-frequency energy from bass-heavy instrumentation. Pop and country looked visually similar which later showed up as confusion between those two classes in the validation results.

3.3 Audio Preprocessing

All audio was resampled to 16kHz. Each audio clip was fixed at 12 seconds (192,000 samples). Clips longer than 12 seconds were randomly cropped and shorter ones were zero-padded. After mixing stems, the audio was normalized using `librosa.util.normalize`.

3.4 Data Augmentation

The key insight for this competition is that test samples are mashups - stems from different songs mixed together with noise. So I replicated this during training. Instead of always using stems from the same song, during training I randomly sampled each stem from any song in the same genre. This cross-song mixing means the model never sees a clean original track during training and is forced to learn genre-level patterns rather than song-level ones.

Additionally, with a 70% probability, a random ESC-50 noise clip was added to each training sample at an intensity of 0.08. The low weight kept the noise realistic without overwhelming the music signal.

4. Feature Extraction

For the CNN and CRNN models, I used Log Mel Spectrograms computed with `torchaudio`. The configuration was: sample rate 16kHz, FFT size 1024, hop length 512, and 64 mel bins. Taking the log of the mel spectrogram compresses the dynamic range and makes the features more suitable for neural network training.

For the AST model, I used the `ASTFeatureExtractor` from Hugging Face which handles its own preprocessing pipeline - it converts raw waveforms into the 128-bin log mel filterbank features that the AST model expects.

5. Modeling and Experimentation

5.1 Model 1 - CNN (From Scratch)

The CNN was built as a baseline to establish whether simple spatial patterns in mel spectrograms are enough for genre classification. The architecture has three convolutional blocks followed by fully connected layers.

Layer	Output Shape	Details
Input	(1, 64, T)	Log Mel Spectrogram
Conv2d + ReLU + MaxPool	(16, 32, T/2)	16 filters, 3x3 kernel
Conv2d + ReLU + MaxPool	(32, 16, T/4)	32 filters, 3x3 kernel
Conv2d + ReLU + AdaptiveAvgPool	(64, 4, 4)	64 filters, adaptive pool to 4x4
Flatten + Linear + ReLU	(128,)	1024 -> 128
Linear (output)	(10,)	128 -> 10 classes

Training config: Adam optimizer, lr=0.001, batch size 16, 4 epochs, CrossEntropyLoss.

5.2 Model 2 - CRNN (CNN + LSTM)

The CRNN was motivated by the fact that music is inherently temporal. A CNN alone captures local frequency patterns but misses the rhythmic structure that evolves over time. By feeding CNN features into an LSTM, the model can capture how the spectrogram patterns change over the duration of the audio - which is actually quite relevant for genres like reggae (offbeat rhythm) or metal (fast tempo patterns).

Layer	Output Shape	Details
Input	(1, 64, T)	Log Mel Spectrogram
Conv2d + ReLU + MaxPool	(16, 32, T/2)	16 filters, 3x3 kernel
Conv2d + ReLU + MaxPool	(32, 16, T/4)	32 filters, 3x3 kernel
Reshape for LSTM	(B, T/4, 32x16)	Treat time steps as sequence
LSTM	(B, T/4, 128)	hidden=128, 1 layer, batch_first=True
Take last hidden state + Linear	(10,)	128 -> 10 classes

Training config: Adam optimizer, lr=0.001, batch size 16, 4 epochs, CrossEntropyLoss.

5.3 Model 3 - AST (Pretrained + Fine-tuned)

The Audio Spectrogram Transformer (AST) is a transformer-based model pretrained on AudioSet - a large dataset of 2 million 10-second audio clips covering 527 sound classes. The key idea is that transformers use self-attention to capture long-range dependencies across the entire spectrogram, unlike CNNs which only look at local patches. This makes AST particularly well suited for audio classification tasks.

I used the checkpoint MIT/ast-finetuned-audioset-10-10-0.4593 from Hugging Face. The classifier head was replaced to output 10 classes instead of 527. The model was fine-tuned end-to-end (no layers frozen) with:

Hyperparameter	Value	Reason
Optimizer	AdamW	Weight decay support for regularization
Learning rate	2e-5	Low LR to avoid destroying pretrained weights
Weight decay	0.01	Regularization via AdamW
Batch size	4	AST is large - GPU memory constraint
Epochs	4	Enough for convergence without overfitting
Training data	100% (no val split)	Maximize generalization given distribution shift

I trained three separate models using seeds 42, 123, and 999. Each model sees the same data but with different random initialization and different augmentation sampling. The final prediction averages the softmax probabilities from all three models. This ensemble approach reduces variance and pushed the Kaggle score above what any single model achieved.

5.4 Ensemble Inference

At inference time, each test mashup is split into 17 random 12-second segments. For each segment, I run it through all three AST models and average the softmax outputs. Segments with RMS energy below 0.01 are skipped as they are likely silent padding. The final genre prediction is the argmax of the averaged probabilities across all valid segments.

6. Performance and Comparative Analysis

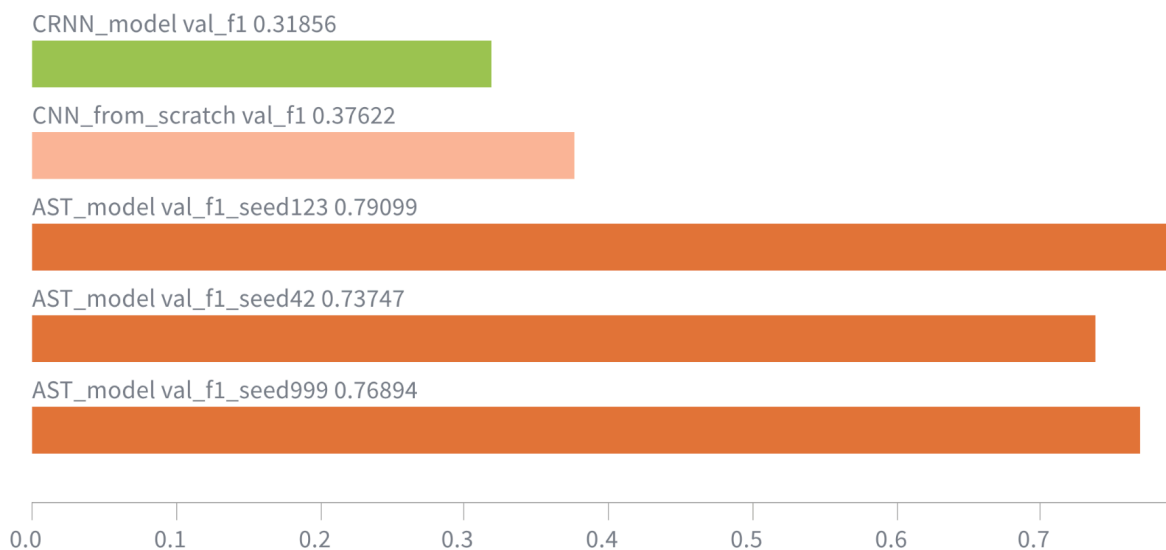
6.1 Evaluation Metric

The primary metric is Macro F1 Score, which computes F1 independently for each of the 10 genre classes and then averages them. This treats all genres equally regardless of how many samples each has. Since the dataset is balanced, Macro F1 and accuracy move together but F1 is still the right choice since it accounts for both precision and recall per class.

6.2 Model Comparison

Model	Val Accuracy	Val Macro F1	Train Loss	Kaggle F1
CNN (from scratch)	43.5%	0.376	1.691	-
CRNN (CNN + LSTM)	35.0%	0.319	1.708	-
AST seed42 (pretrained)	74.5%	0.769	0.406	-
AST seed123 (pretrained)	79.0%	0.791	0.431	-
AST seed999 (pretrained)	77.5%	0.769	0.447	-
AST Ensemble (3 seeds)	-	-	-	0.87237

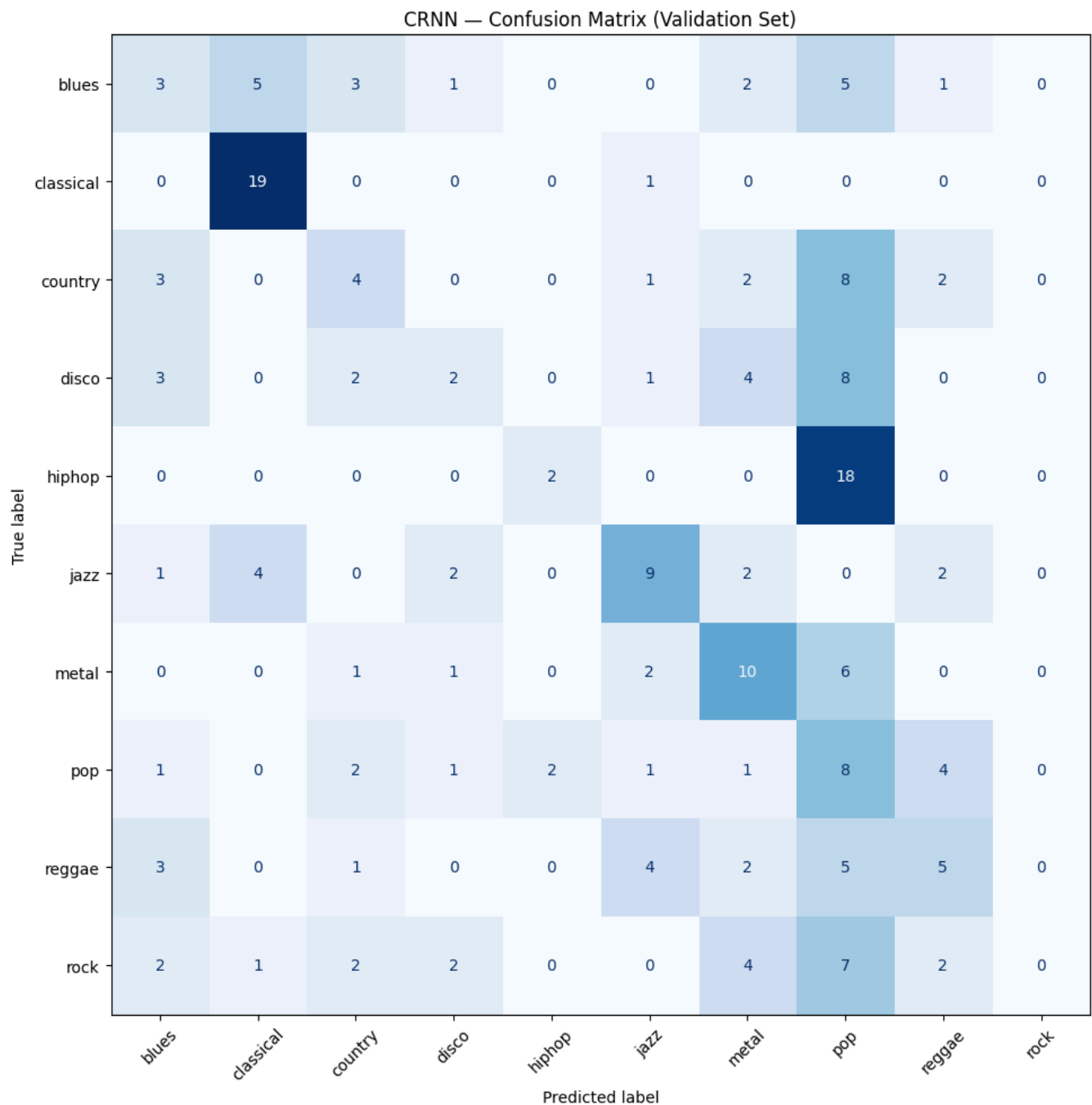
Model Comparison - Validation Macro F1 Score



The gap between CNN/CRNN and AST is significant. CNN got 43.5% validation accuracy while AST seed123 alone hit 79%. This is expected - AST has been pretrained on 2 million audio clips and already understands rich audio representations. The CNN is learning from scratch on only 800 training samples, which is not enough to learn robust genre-level features especially when the test distribution is so different.

One thing worth noting is that CRNN actually performed worse than CNN here. The LSTM added model complexity but with only 4 epochs of training on limited data, it did not have enough time to learn useful temporal dependencies. With more epochs and more data it would likely outperform the plain CNN.

6.3 Error Analysis



Looking at the confusion matrix from the CRNN validation run, the most common misclassifications were between rock and metal, and between country and pop. This makes intuitive sense - rock and metal share similar instrumentation (electric guitars, drums) and the main difference is intensity/distortion which gets washed out in noisy mashups. Country and pop both tend to be vocal-driven with melodic structure, making them hard to separate when stems are mixed across songs.

Classical was generally the easiest to classify correctly, probably because its spectral profile is so distinct from other genres - sparse energy, dynamic variation, no heavy bass or distortion.

6.4 W&B Tracking

All four runs were tracked in the W&B project DL GENAI-PROJECT-T12026:

Run Name	Metrics Logged
CNN_from_scratch	train loss, val accuracy, val F1 per epoch
CRNN_model	train loss, val accuracy, val F1 per epoch
AST_model	train loss per seed (seed42, seed123, seed999)
final_results	Kaggle Macro F1 = 0.87237, inference time, num test samples

7. Deployment

The AST model (seed123, best single-model F1) was deployed as a Gradio app on Hugging Face Spaces at 24f1000556/messy-mashup-genre-classifier. Users can upload any .wav file and the app returns the predicted genre along with a confidence bar chart showing probabilities across all 10 classes. The inference uses 10 random segments per file with the same silent segment filtering as the Kaggle submission.

8. Conclusion and Future Work

8.1 Key Learnings

- Pretrained models make a huge difference when training data is limited - AST with 4 epochs outperformed CNN trained from scratch by a wide margin
- Distribution shift is the real challenge here - augmentation that matches the test distribution (cross-song mixing, ESC-50 noise) was more important than model architecture choices
- Training on 100% of data without a validation split made sense given the small dataset size and the distribution gap between train and test
- Ensemble of multiple seeds is a reliable way to squeeze out extra performance without changing anything about the model architecture

8.2 Challenges

- GPU memory constraint with AST - batch size had to be kept at 4 which made training slow
- No clean validation signal for AST since val distribution also doesn't match test distribution
- Kaggle session timeout risk when training 3 x 4 epochs sequentially

8.3 Future Improvements

- Try HuBERT or larger audio transformers - they have more parameters and richer pretrained representations
- Experiment with longer fine-tuning (10-15 epochs) with a learning rate scheduler
- Add more aggressive augmentation - pitch shifting, time stretching, stem dropout
- Try weighted ensemble instead of equal-weight averaging across seeds
- Use all 4 stems separately as input channels instead of mixing them before inference

9. References

- Gong et al. (2021). [AST: Audio Spectrogram Transformer.](#)
- Piczak, K.J. (2015). [ESC: Dataset for Environmental Sound Classification.](#)
- McFee et al. [Librosa: Audio and Music Signal Analysis in Python](#)
- [Hugging Face Transformers Library.](#)
- [PyTorch.](#)
- [Weights & Biases.](#)